```
for (int i=0; i<250; i+=5) {
  digitalWrite(13, HIGH);
  delay(i);
  digitalWrite(13, LOW);
  delay(i);
}
```

This sample code uses multiple `for` loops to vary the speed at which the LED on pin 13 blinks. Beginning with a quarter second on and a quarter second off, the first `for` loop will decrement the counter variable `i` until it hits 0. Using this variable in our `delay()` function, we can speed up and slow down how quickly the LED blinks.

delayMicroseconds()

Rather than a long delay, we can use the `delayMicroseconds()` function to delay for a much shorter time like we did in Chapter 5. As with `delay()`, there is not much to its syntax, as follows:

```
delayMicroseconds(time)
```

Unlike `delay()`, time here is specified in microseconds, or millionths of a second, where a time period of 1000 microseconds would equal 1 millisecond or 0.001 of a second, 10,000 would equal 10 milliseconds or 0.01 of a second, and so on. While the value for time is specified as a long integer, it is only known to work up to a maximum delay value of 16,383. Likewise, delays below 3 microseconds don't work reliably either. Instead, use the `delay()` function for reliable operation for anything over a few thousand microseconds.

Like we did in the Noisy Cricket project to create a different tone, it is possible to use `delayMicroseconds()` to create a low-tech PWM for pins without PWM available to them. Take the following sample code:

```
digitalWrite(13, HIGH);
delayMicroseconds(100);
digitalWrite(13, LOW);
delayMicroseconds(900);
```

By turning on and off the pin very quickly it is possible to simulate a dim LED, just as we did with PWM except that this time it is on pin 13. In this sample code, in the time span of 1 millisecond, we turn on the LED for 100 microseconds and off for 900 microseconds for a 10% duty cycle resulting in a fairly dim LED.

So that's two simple methods for creating a delay using the built-in Arduino functions; however, the problem with delay functions in the Arduino library is that nothing else can run while the delay is in effect. Imagine a `while` loop that cycles through for as many milliseconds as we specify and does nothing else other than check the current time until the delay time has passed. Of course, that's not entirely true as some hardware-based functions will still function normally including PWM, hardware serial, and hardware interrupts. Regardless, there is no reason for the Arduino just sitting there doing nothing when we can use it for other things. To create a delay without a delay function we can use something called a hardware timer.

millis()

Inside the microcontroller on the Arduino board there are three on-board hardware timers that work in the background to handle repetitive tasks like incrementing counters or keeping track of program